



WAREHOUSE SLOTTING OPTIMIZER

Optimizing storage-slot assignment using co-pick pattern mining and graph-based allocation

Project Report — Data Structures and Algorithms-3 (25CS2103E)

Gatik Sharma (2520090223) | Mekonda Abhi Sai Sagar (2520090173)

Team 17 • Section 10

Under the guidance of K. Chandusha, Asst. Prof., CS&IT

Hyderabad, Telangana, India • A.Y. 2026–2027

Abstract



Core Idea

Assign products to storage slots based on how often they're ordered together — cutting picker travel distance and speeding up fulfillment.

Course

Data Structures & Algorithms-3

Team

Team 17 · Section 10

The Warehouse Slotting Optimizer improves warehouse efficiency by assigning products to optimal storage locations based on historical order data. It analyzes customer orders to identify products frequently picked together using co-pick pattern mining, then applies optimization techniques to place related items in nearby slots — minimizing picker travel distance and reducing order fulfillment time.

The project pipeline includes **data preprocessing** to clean SKU and order records, **affinity generation** between products, and an **optimized slot-allocation algorithm** built on graph-based techniques.

The solution enhances warehouse throughput, improves space utilization, and increases operational efficiency while adapting to changing order patterns — demonstrating a practical, real-world application of Data Structures and Algorithms.

Introduction

Warehouses depend on how efficiently pickers move to fulfill orders. When frequently co-ordered items sit far apart, travel time balloons and throughput suffers — a problem that only grows as order volumes and catalogs scale.



The Problem

Poor slotting forces pickers to travel long, inefficient paths, inflating order fulfillment time.



The Insight

Historical order data reveals which products are habitually picked together — a pattern worth exploiting.



The Approach

Model products and their co-pick affinity as a graph, then optimize slot placement so related items sit close together.

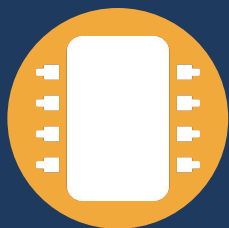


The Outcome

Shorter picker paths, better space utilization, and a system that adapts as order patterns shift.

Hardware Requirements

Minimum system configuration to develop, test, and run the optimizer



Processor

Intel i5 / AMD Ryzen 5 (or equivalent), quad-core,
2.4 GHz+



RAM

8 GB minimum (16 GB recommended for larger
order datasets)



Storage

256 GB SSD — for datasets, dependencies, and
generated slot maps



Network

Stable internet connection for package installs and
dataset access

Software Requirements

Tools and stack used to build the slotting engine



Operating System

Windows 10/11, macOS, or Linux



Programming Language

Python 3.10+ for data processing and algorithm logic



IDE / Editor

VS Code / PyCharm / Jupyter Notebook



Data & Libraries

Pandas & NumPy for preprocessing, NetworkX for graph-based affinity modeling, Matplotlib for slot-map visualization

Methodology

From raw order data to an optimized slotting layout

1

Data Collection & Preprocessing

Gather historical order/SKU records; clean, deduplicate, and standardize product and transaction data.

2

Co-Pick Pattern Mining

Apply frequent-itemset mining (Apriori / FP-Growth) to find products that are habitually ordered together.

3

Affinity Graph Construction

Model products as nodes and co-pick strength as weighted edges, forming a product-affinity graph.

4

Slot Allocation Algorithm

Use graph-based clustering/optimization to assign highly-affine products to nearby warehouse slots.

5

Evaluation & Validation

Simulate picker routes on the new layout and compare travel distance and fulfillment time against the baseline.

Literature Summary

Prior work informing the slotting and pattern-mining approach

Reference	Focus Area	Key Contribution
Duque-Jaramillo et al. (2024)	Correlated storage assignment	Sorting-based slotting model combining space assignment and picking strategy via a two-phase mixed-integer approach
Schiraldi et al.	Slot-code optimization	Quantifies travel-distance and handling-time reduction from optimal vs. uniform item allocation in order picking
Affinity-Based Slotting in Warehouses with Dynamic Order Patterns	Affinity-based storage assignment	Extends the classic Storage Location Assignment Problem to a multi-period model and evaluates affinity-based rules under shifting demand
Agrawal & Srikant (1994)	Association rule mining	Introduces the Apriori algorithm for mining frequent itemsets — the basis for detecting co-pick patterns
Han, Pei & Yin (2000)	Frequent pattern mining	Proposes FP-Growth, mining frequent patterns without candidate generation — a faster alternative to Apriori
Zhang (2016)	Picker-to-parts strategies	Develops correlated storage strategies linking product correlation to picking-route efficiency



Gap identified: existing work optimizes slotting or affinity separately — few combine graph-based co-pick mining with a data-structure-driven allocation algorithm, which this project addresses.